Deep Learning Project

Kirk Kennedy Lincoln
Woolf University

MS in AI - Masters Capstone Module 4

Deep Learning Project

Report Overview

During our research of the Azure VM LLM Inference Dataset https://azurepublicdatasettraces.blob.core.windows.net/azurellminferencetrace we addressed the machine learning problem of time-series sequence prediction for high-performance compute environments (Microsoft Azure 2024). We analyzed two parts of this dataset in terms of *conversations* and *code completions*; subsequently, we will refer to these subsets as *conv* and *code*, respectively. When exploring code completion (bug fixes, hot-patches) vs. natural language hallucinations, Liu, C., Lin, H. Y., & Thongtanunam, P. (2025) discovered that code completions contain a ~30% increase over their natural language counterpart (conversational). The format of Liu, C. et al's natural language conversation generation was primarily git commit messages; therefore, we will consider this as strong supporting evidence for our goals and choices (Liu, C. et al., 2025). The deep learning task we are addressing is the RNN application to time-series data utilizing a classical LSTM from PyTorch. Afterwards, we perform an experimental comparison using a TransformerEncoder, which is applied as an encoder-only architecture using a linear projection head.

Dataset and Task Description

Our dataset consists of two data files: *AzureLLMInferenceTrace_conv_1week.csv* and *AzureLLMInferenceTrace_code_1week.csv* with code being discarded after our initial EDA processing. The *conv* dataset ranges from 2024-05-12 to 2024-05-19 and *code* is in the range of 2024-05-10 to 2024-05-17. Each file contains three fields:

- *TIMESTAMP*
- *ContextTokens*
- *GeneratedTokens*

All timestamps are formatted into ISO8601 datetimes from a raw string and resolves to millisecond steps. *ContextTokens* can be considered "input tokens" and *GeneratedTokens* could otherwise be

classified as “output tokens”. We should acknowledge our evaluation relies on a single dataset source (Azure LLM Inference Traces), which Roque et al. (2025) identify as a significant risk in time-series forecasting research. What their analysis demonstrates is that out of four cherry-picked datasets, 46% of forecasting models could appear to be the best performer, and that deep learning approaches show higher sensitivity to dataset selection than classical methods; subsequently, our study is subject to this limitation. The LSTM's superior performance over the Transformer encoder on this particular trace does not guarantee the same ranking would hold on other workload distributions, such as the M4 or ETT benchmarks commonly used in forecasting literature. However, our goal’s scope is non-inclusive of a “general time-series prediction model”, which frees us of any academic constraints portrayed as fundamental issues.

Our overall goal, to train a deep learning model to predict time-series sequence data for high-performance computing environments, pushes us to ignore the tokens for input and output. Our training/validation/testing split is solely timestamp data with the “meta” context of requests per second for an LLM service is incurring some correlative trend of input/output tokens. Therefore, we resampled the database into a size of *one second* producing a univariate time-series of *requests per second*. The choice between *code* and *conv* came down to no zero-bins. In computing, zero-bins are either associated with an artificial measurement (log scaled outputs with zero-allowed), or complete downtime. Since we are looking for a model that has the potential to work with micro-VM tools, we need to maintain a baseline *above* zero, which *conv* fulfills this requirement. Our dataset splitting is as follows: 60/20/20 split yielding approximately 362880/120960 /120960 one-second bins (~4.2/~1.4/~1.4 days).

Conv modeled a normal distribution (skew 0.27) opposed to the right-skewed (skew 1.14), weak bi-modal trend we saw with the *code* dataset. Therefore, we avoid the zero-bin overpowering, collect normally distributed data, and match the equivalent range of the *code* dataset (0-146 r/ps) to the *conv* dataset (5-176 r/ps).

Model Architecture and Design Decisions

Initially, we needed to do some math before diving into the modeling or approach. We need to identify the neighborhood of our error findings to determine whether we are in the right range of outputs. The error finding contains four methods two of which are heavily recommended by Hyndman, R.J. & Anthanasopoulos, G. (2018):

- MSE - how wrong are we on average (quadratic error, punish large gaps)
- MAE - how wrong are we on average (absolute error, no squaring)
- Persistence - whatever happened last second will happen this second
- Rolling Mean - predict the average output for a chosen length of time

Following these calculations, there were two selections we made during this process, nn.LSTM and nn.TransformerEncoder, which uses nn.TransformerEncoderLayer(s). In order to prepare the data for ingestion, we needed to create a "windowing" function, which helped us create a sequence of bigrams based upon three parameters:

- *step* - size of primary sequence
- *horizon* - size of secondary sequence
- *stride* - number of timesteps to advance between each bigram sequence

After we discovered how to package our DataLoaders (numpy arrays) and initiate a training loop, we selected functions to best suit an LSTM architecture. Dropout is always a strong contender for any type of model because it prevents co-adaptation in stochastic neuron retention; consequently, we chose 20% dropout, which is considered an optimal baseline in the industry (Srivastava, N. et al, 2014, pg. 1933). The LSTM layer was chosen as advised by Hochreiter, S., & Schmidhuber, J. (1997) since our target was less than 1000 discrete-time steps in each sequence bigram; moreover, we stacked twice (hyperparameter num_layers=2), and nn.Linear was chosen as our fully-connected layer. The *forward* loop implements an unsqueeze so we can fit into the LSTM layer, the next step x = x[:, -1, :] grabs the hidden layer summary of what the LSTM layer just saw. Afterwards dropout regulates training by randomly zeroing 20% of the

LSTM's hidden state on each forward pass, preventing the network from over-relying on any single feature. The fully-connected layer then projects the 128-dimensional hidden state down to 10 values, our forecast horizon.

Experimental Comparison

The following model, nn.TransformerEncoder, allows us to balance the difference between LSTM gates and recurrent units, versus an attention-focused, parallel-executing encoder model. Our transformer uses encoder layers sandwiched between an embedding layer and a fully-connected output layer. Similar to the LSTM, we start out with unsqueezing an extra dimension onto the input, for a shape that our Linear embedding layer can ingest; in addition, our embedding layer is multiplied against the square root of our embedding layers “out_features”. Referred to as embedding scaling by Vaswani et al. (2017, Section 3.4), this operation onto the square root of d_model (out_features), which smooths out the magnitude to ease the strain on the attention layer softmax calculation. Moreover, we take the hidden layer and process the 32-dimensional hidden state down to 10 values much like our LSTM. One thing to note is that *dropout* is being managed by the TransformerEncoder as a hyperparameter, which applies it before and after the self-attention and feedforward sublayers.

Results and Interpretation

When analyzing the two architectures we see results that are comparable, which produces similar error outcomes; however, the parameters are exponentially larger in the LSTM category, and the training time is 50% longer than its Transformer counterpart:

Metric	LSTM	Transformer
Test MSE	38.51	41.84
Test MAE	4.90	5.12
Parameters	200458	8938
Training time	93.028069 s	66.504246s

Since we chose two layers for the LSTM architecture our parameters field blew up in terms of how the gates expanded our hidden layers. A seasoned engineer would point to the fact that our LSTM used two layers but the Transformer only used one; however, our training results did not seem to improve marginally after ~2 epochs. In this case, the problem isn't having enough compute power to train our models in an efficient manner, but we don't have enough data (or diverse enough data) to merit a harder train. However, our validation set seems to follow our training curve moderately well, which would indicate there is no memorization or overfitting taking place.

This issue becomes apparent when viewing our “quiet” test:

=== quiet ===
Last 10s input: ['1.0', '0.0', '1.0', '0.0', '1.0', '0.0', '1.0', '0.0', '1.0', '0.0']
Next 10s forecast lstm: ['7.80', '6.75', '6.48', '6.52', '7.33', '7.02', '7.19', '7.00', '8.34', '8.24']
Next 10s forecast transformer: ['16.09', '15.69', '15.75', '16.02', '17.07', '16.43', '16.18', '16.35', '17.55', '15.94']
Forecast mean lstm_model: 7.27, max: 8.34
Forecast mean transformer_model: 16.31, max: 17.55

Although the transformer performed less optimally than our LSTM, we observe the issue of the transformer not understanding a “soft minimum”, which was on average around 5.0 in its training data. Instead of scaling the outputs to accommodate the low-velocity churn, it defaults to using the training set’s mean.

Both models default toward the training distribution's central tendency rather than tracking the actual temporal pattern. The LSTM lands closer to the input range (mean 7.27 vs actual 0 to 1), while the Transformer overshoots significantly (mean 16.31), suggesting it is more susceptible to mean-reversion. Neither model generalizes to out-of-distribution workload patterns, which is important for the scope of our goals; moreover, production systems would need to handle novel load profiles that weren't present in training data.

Limitations and Risks

The headlining limitation of our dataset is the fact it’s a single table, which Roque, L. et al. (2025) suggests is deficit when selecting data. However, our model is designed to find patterns in sequences, which pertain to virtual machine computing environments. Since this is the case additional data would introduce tags outside of a list of timestamps to add into our scaling in-and-out model. Another limitation to our training regiment is within our EDA, which forgoes the analysis of input and output tokens. If we spent time building functions that tracked and preprocessed rolling means for those values, we could have a much richer dataset to train with. The main obstacle implementing this was our constraint on time and

ensuring the calculations are done correctly, we can possibly implement this into our synthesis project since it would be a helpful supporting feature.

When it comes to our model architectures, they are extremely simple; moreover, the transformer lacks a positional encoder, we use "*dim_feedforward=input_size*2"*, both of which really limit the TransformerModel class (Vaswani et al., 2017, Section 3.5). The last short fall of this training regiment is the fact we didn't have enough compute time to analyze models with 5, 10, 20, etc. steps in its window function. Using 300 as our original input was a logical step towards "5 minutes"; however, it was not traced and proven to be the optimal (or one of the optimal) time bucket options.

Ethical and Responsible Use

The greatest concern LLMs and AI companies are exposing is the usage of resources needed for their energy. AI infrastructure represents one of the fastest-growing demands on energy resources in order to increase our knowledge. Our world has never seen individuals posing world-wide standards and metric evaluations all for a singular technology. We must be aware of the wastefulness that any corporation will incur to achieve their goals.

When it comes to ethics, the concern is not our "daily use", which could pose a decrease in environmental sustainability heading into the future, but the greatest upfront risks are the inefficiencies current computing environments impose. Scale agentic AI use fueled by large investment firms, we can quickly see ourselves painted into a corner needing to sacrifice our precious environment on the altar of technology in the name of AI. If our LSTM or Transformer model fails to predict future sequences in scaling accurately, then our systems will either perform badly (underprovisioned compute) or become extremely wasteful (overprovisioned compute). Additionally, any data recorded by the AI system could retain temporal patterns that could be considered proprietary in nature.

With this in mind we must focus on scheduling of computational resources, which is what Stojkovic, J. et al (2025) is accomplishing with their creation of DynamoLLM, which each performance metric they meet surpasses what we consider "green"; moreover, under their experimental conditions 52% of energy

is conserved, 38% of operational carbon emissions are mitigated, and costs towards the customer are reduced by 61%. DynamoLLM claims to meet all SLO's within those latency requirements. If this is the case, then we are encroaching on a territory when "energy conservative AI open source software" will begin to trend and take over the space.

Future Improvements

The single number one improvement would probably be a larger dataset in the same format, that would provide longer-term patterns for these models to generalize from. However, there is one caveat, that we perform stair-stepping increases in our preprocessed data (increasing step parameter in windowing function), which will provide our own temporal data patterns in regards to model performance. Secondly, we should implement a rolling calculation of the input/output tokens, but window those features so the LSTM/Transformer can recognize the "requests per second" more easily. In all reality, focusing on requests per second is the naive implementation, with token context being the true feature set that will train this model properly. We were not able to engineer this feature due to its complexity and high probability of failure on the initial attempt. Finally, we should implement a PositionalEncoder class into the Transformer in order to fully implement the idea of a transformer Vaswani, A. et al (2017).

References

Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and Practice* (2nd ed.). OTexts.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

Liu, C., Lin, H. Y., & Thongtanunam, P. (2025). Hallucinations in code change to natural language generation: Prevalence and evaluation of Detection Metrics. *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, 2538–2560. https://doi.org/10.18653/v1/2025.ijcnlp-long.137

Microsoft Azure. (2024). *Azure Public Dataset: AzureLLMInferenceDataset2024* [Data set]. GitHub. https://github.com/Azure/AzurePublicDataset

Roque, L., Cerqueira, V., Soares, C., & Torgo, L. (2025). Cherry-picking in time series forecasting: How to select datasets to make your model shine. *Proceedings of the AAAI Conference on Artificial Intelligence*, *39*(19), 20192–20199. https://doi.org/10.1609/aaai.v39i19.34224

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929–1958.

Stojkovic, J., Zhang, C., Goiri, Í., Torrellas, J., & Choukse, E. (2025). DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)* (pp. 1348–1362). IEEE. https://doi.org/10.1109/HPCA61900.2025.00102

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., N.Gomez, A., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need. Advances in Neural Information Processing Systems, 30*. https://arxiv.org/abs/1706.03762